Day 1 – Lab 1.2: Build and Run an HTTP Function Locally

Module 2 · Local Development Toolchain

Objective: Build a C# HTTP-triggered Azure Function that accepts order details, calculates the order total and returns a JSON response.

Business scenario: Contoso Orders needs an API through which customers can submit order quantities and unit prices to receive an order quote. The API must reject invalid requests instead of processing incomplete data.

1. Prerequisites

Before starting, verify the following:

- Lab 1.0 development environment is ready.
- .NET 10 SDK and Azure Functions Core Tools 4.x are installed.
- VS Code and C# Dev Kit are available.
- Azurite extension is installed and enabled.
- Lab 1.1 Function App has been provisioned.

Install the REST Client VS Code extension to execute HTTP requests directly from a `.http` file.

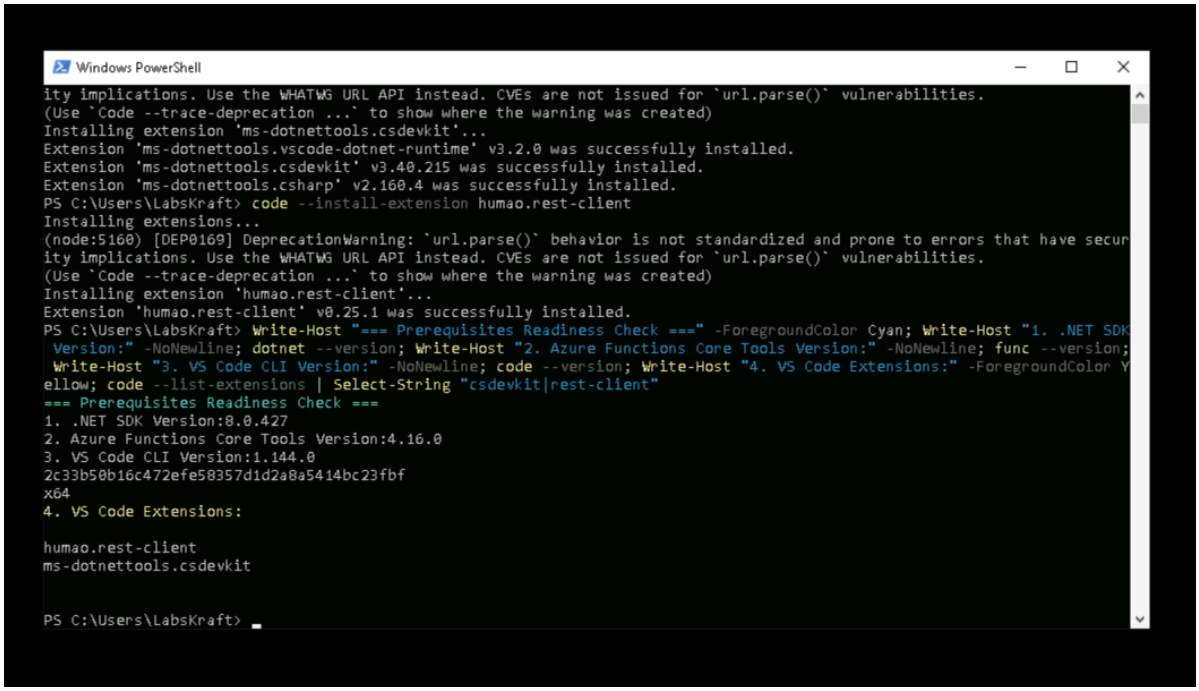
Install REST Client extension

Note: This lab runs locally. We will not deploy the Function App created in Lab 1.1 yet.

PowerShell

None

```
Write-Host "=== Prerequisites Readiness Check ===" -ForegroundColor Cyan; Write-Host "1. .NET SDK Version:" -NoNewline; dotnet --version; Write-Host "2. Azure Functions Core Tools Version:" -NoNewline; func --version; Write-Host "3. VS Code CLI Version:" -NoNewline; code --version; Write-Host "4. VS Code Extensions:" -ForegroundColor Yellow; code --list-extensions | Select-String "csdevkit|rest-client"
```



2. Create the Function Project

Step 1 – Open VS Code

1. Launch VS Code.
2. Select Terminal → New Terminal.
3. Navigate to your preferred working directory.

Step 2 – Create the project

Execute:

```shell
func init contoso-fn --worker-runtime dotnet-isolated
--target-framework net10.0
```

This creates a C# Azure Functions isolated-worker project targeting .NET 10.

```
Troubleshooting steps if you get .NET version error

Run this simplified script in your Administrator PowerShell window to
download and install the .NET 10 SDK:

[Net.ServicePointManager]::SecurityProtocol = 3072
Invoke-WebRequest -Uri "https://dot.net/v1/dotnet-install.ps1"
-OutFile "$env:TEMP\dotnet-install.ps1"
& "$env:TEMP\dotnet-install.ps1" -Channel 10.0 -InstallDir
"C:\Program Files\dotnet"

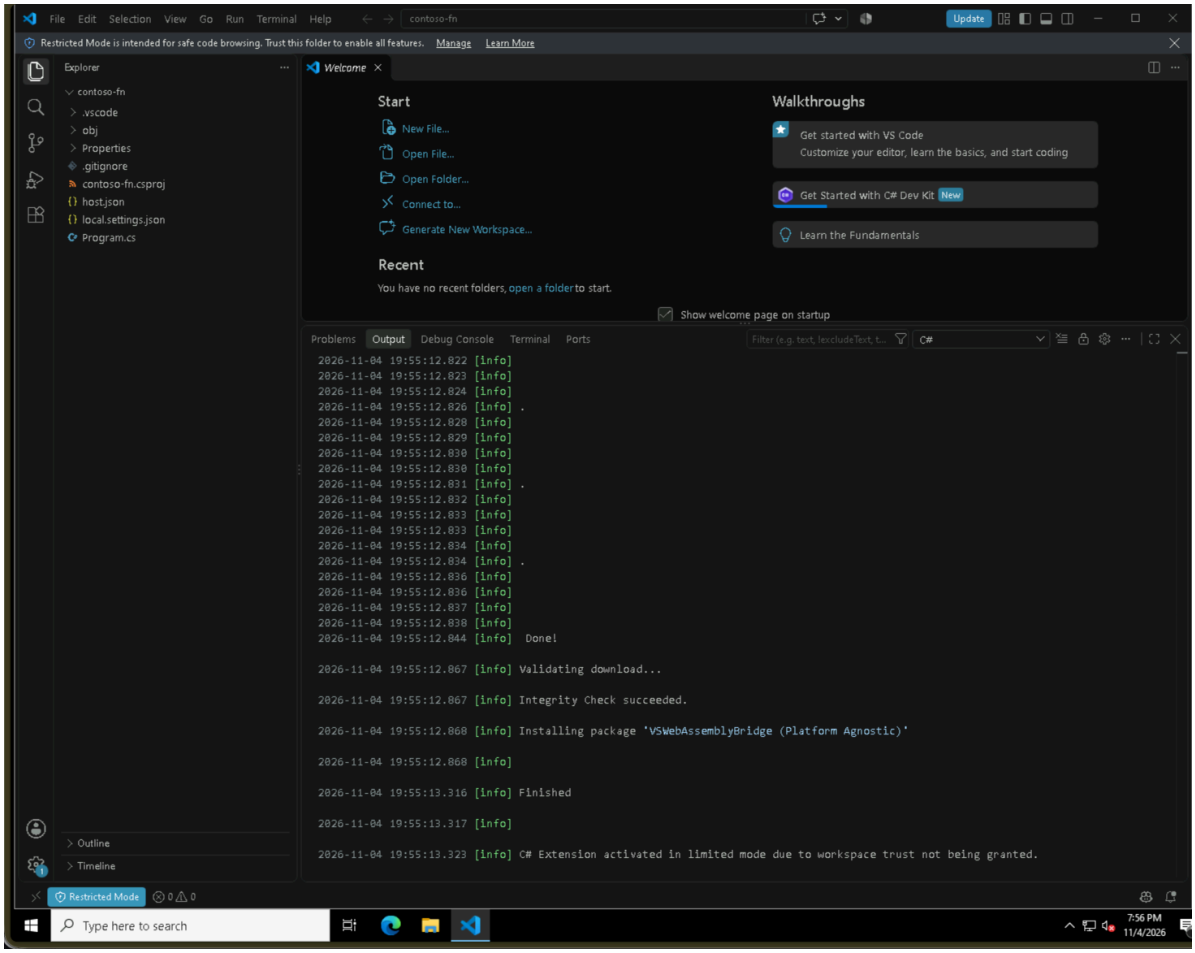
After the installation finishes, update your current session's PATH
environment variable and verify:

$env:Path += ";C:\Program Files\dotnet"
```

```
PS C:\Windows\system32> [Net.ServicePointManager]::SecurityProtocol = [Net.ServicePointManager]::SecurityProtocol -or [N
et.SecurityProtocolType]::Tls12; Invoke-WebRequest -Uri "https://dot.net/v1/dotnet-install.ps1" -OutFile "$env:TEMP\dotn
et-install.ps1"; & "$env:TEMP\dotnet-install.ps1" -Channel 10.0 -InstallDir "C:\Program Files\dotnet"
Exception setting "SecurityProtocol": "Cannot convert value "True" to type "System.Net.SecurityProtocolType". Error:
"Invalid cast from 'System.Boolean' to 'System.Net.SecurityProtocolType'.""
At line:1 char:1
+ [Net.ServicePointManager]::SecurityProtocol = [Net.ServicePointManage ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], SetValueInvocationException
    + FullyQualifiedErrorId : ExceptionWhenSetting

dotnet-install: Remote file https://builds.dotnet.microsoft.com/dotnet/Sdk/10.0.403/dotnet-sdk-10.0.403-win-x64.zip size
 is 300745255 bytes.
dotnet-install: Downloaded file https://builds.dotnet.microsoft.com/dotnet/Sdk/10.0.403/dotnet-sdk-10.0.403-win-x64.zip
size is 300745255 bytes.
dotnet-install: The remote and local file sizes are equal.
dotnet-install: Extracting the archive.
dotnet-install: Note that the script does not ensure your Windows version is supported during the installation.
dotnet-install: To check the list of supported versions, go to https://learn.microsoft.com/dotnet/core/install/windows#s
upported-versions
dotnet-install: Installed version is 10.0.403
dotnet-install: Installation finished
PS C:\Windows\system32>
PS C:\Windows\system32> $env:Path += ";C:\Program Files\dotnet"
>> dotnet --version
10.0.403
```

Step 3 – Open the project `contoso-fn` created in previous step in vs code.



Step 4 – Verify project structure

You should see the following files:

```
None
contoso-fn/
|
├── Program.cs
├── host.json
├── local.settings.json
```

```
├── .gitignore
└── contoso-fn.csproj
```

The project uses `Program.cs` as its application entry point, while `host.json` contains Functions host configuration.

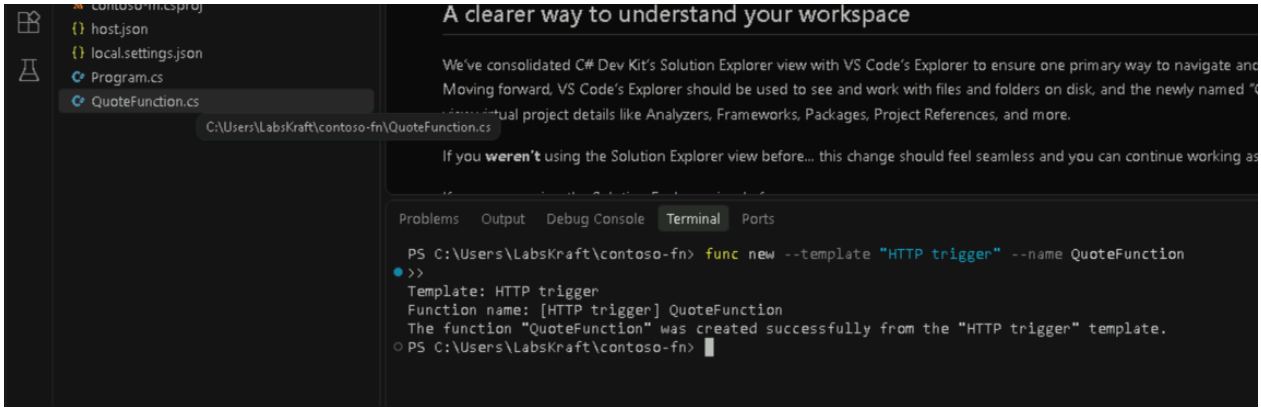
3. Create the HTTP Trigger

Step 1 – Generate the Function

Open the VS Code terminal and execute:

```
None
func new --template "HTTP trigger" --name
QuoteFunction
```

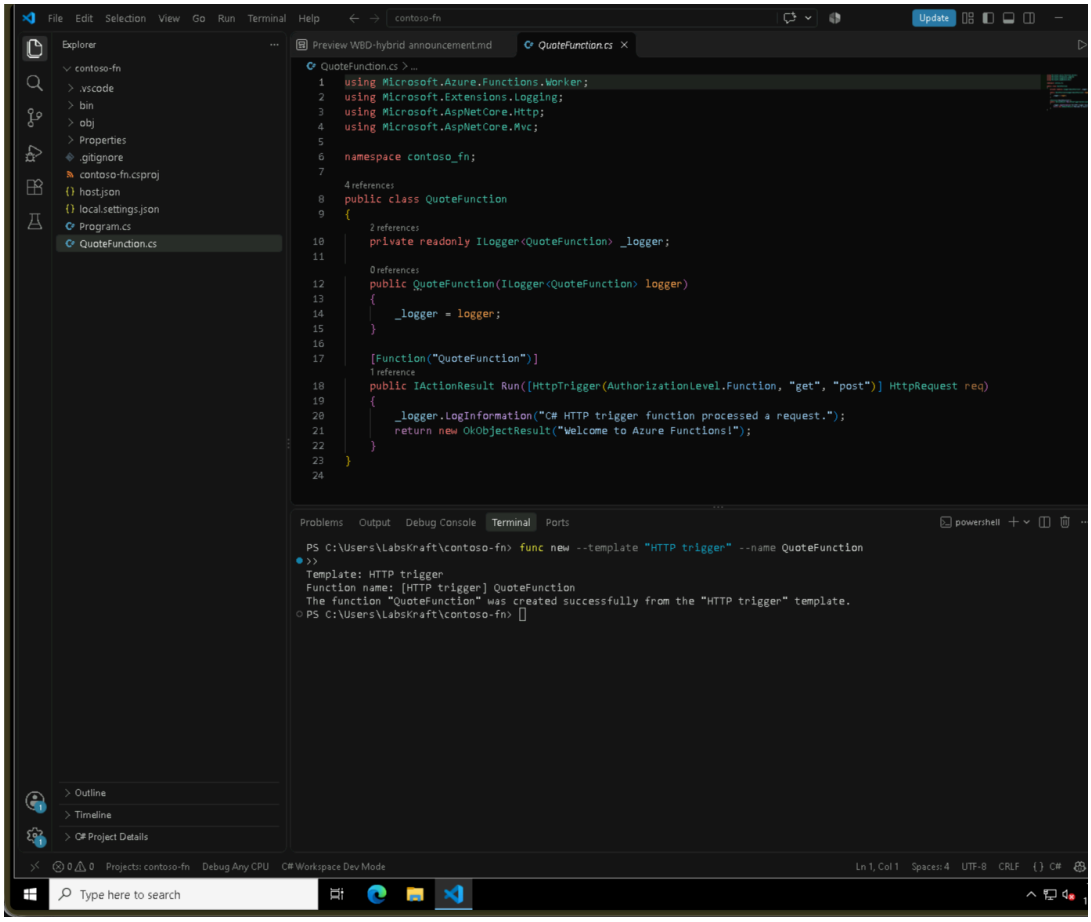
If prompted for authorization level, select Function.



Step 2 – Verify the generated file

A new C# file named `QuoteFunction.cs` should appear in the project.

Step 3 – Open [QuoteFunction.cs]



Replace the generated code with the following implementation.

4. Implement the Order Quote API

The API will accept:

- `qty` – Number of items ordered.
- `unitPrice` – Price per item.

File: `QuoteFunction.cs`

Replace the complete file contents:

None

```
using System.Net;
using System.Text.Json;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;

namespace ContosoFn;

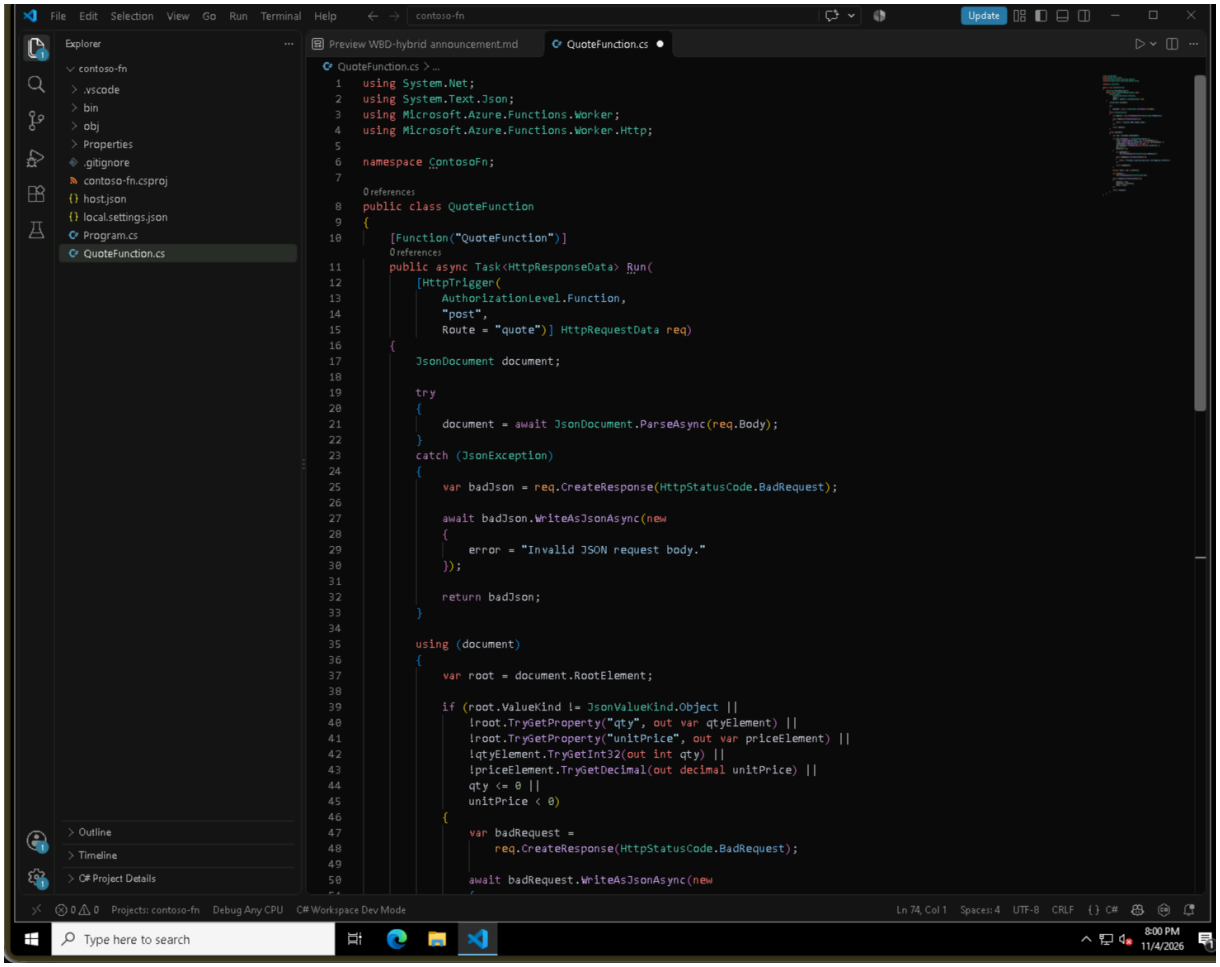
public class QuoteFunction
{
    [Function("QuoteFunction")]
    public async Task<HttpResponseData> Run(
        [HttpTrigger(AuthorizationLevel.Function,
"post", Route = "quote")] HttpRequestData req)
    {
        JsonDocument document;
        try
        {
            document = await
JsonDocument.ParseAsync(req.Body);
        }
        catch (JsonException)
        {
            var badJson =
req.CreateResponse(HttpStatusCode.BadRequest);
            await badJson.WriteAsJsonAsync(new
            {
                error = "Invalid JSON request body."
```

```
            });
            return badJson;
        }

        using (document)
        {
            var root = document.RootElement;
            if (root.ValueKind !=
JsonValueKind.Object ||
                !root.TryGetProperty("qty", out var
qtyElement) ||
                !root.TryGetProperty("unitPrice", out
var priceElement) ||
                !qtyElement.TryGetInt32(out int qty)
||
                !priceElement.TryGetDecimal(out
decimal unitPrice) ||
                qty <= 0 ||
                unitPrice < 0)
            {
                var badRequest =
req.CreateResponse(HttpStatusCode.BadRequest);
                await badRequest.WriteAsJsonAsync(new
                {
                    error = "Provide a positive qty
and a non-negative unitPrice."
                });
```

```
                return badRequest;
            }

            decimal total = qty * unitPrice;
            var response =
req.CreateResponse(HttpStatusCode.OK);
            await response.WriteAsJsonAsync(new
            {
                quantity = qty,
                unitPrice = unitPrice,
                total = total
            });
            return response;
        }
    }
}
```



Understanding the implementation

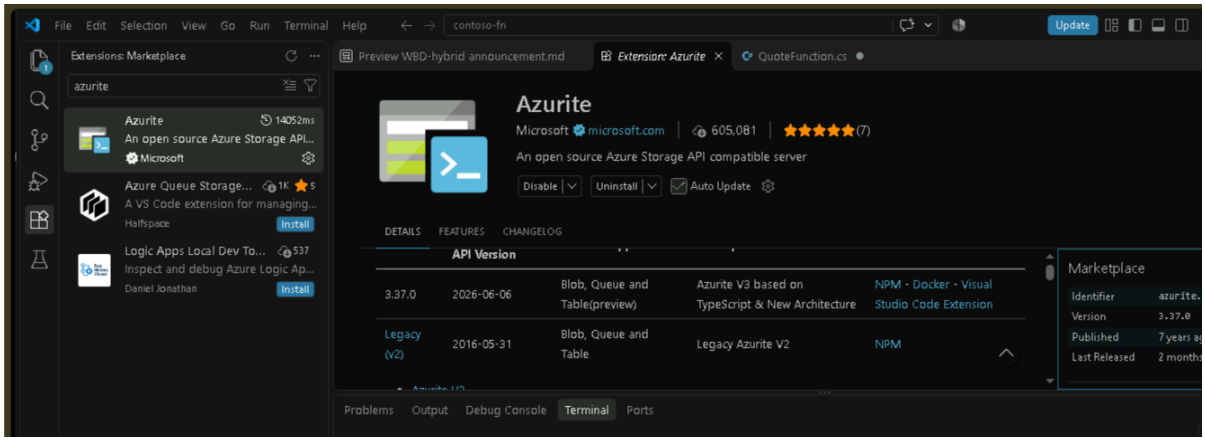
- `[Function]` identifies the Function to the Azure Functions runtime.
- `[HttpTrigger]` configures the HTTP POST endpoint.
- `AuthorizationLevel.Function` requires a Function key when hosted in Azure.
- `JsonDocument` reads the incoming JSON.
- `TryGetProperty()` checks whether the required fields exist.
- Invalid input returns HTTP 400.
- Valid input returns HTTP 200 with the calculated total.

5. Configure Local Storage

The Functions host requires a storage configuration, even though this particular HTTP Function does not directly access Storage.

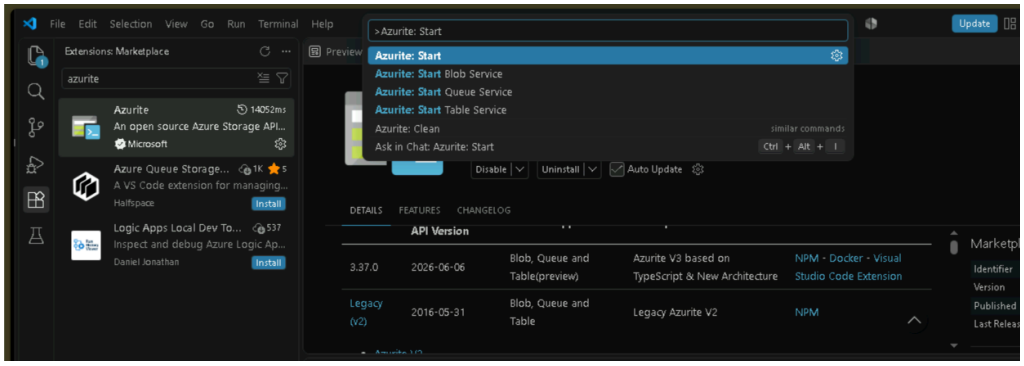
For local execution, we will use Azurite.

Step 1 – Install Azurite Extension in VS Code



Step 2 – Start Azurite

1. Open the VS Code Command Palette using `Ctrl + Shift + P`.
2. Search for `Azurite: Start`.
3. Select the command.
4. Confirm that the Azurite services start successfully.

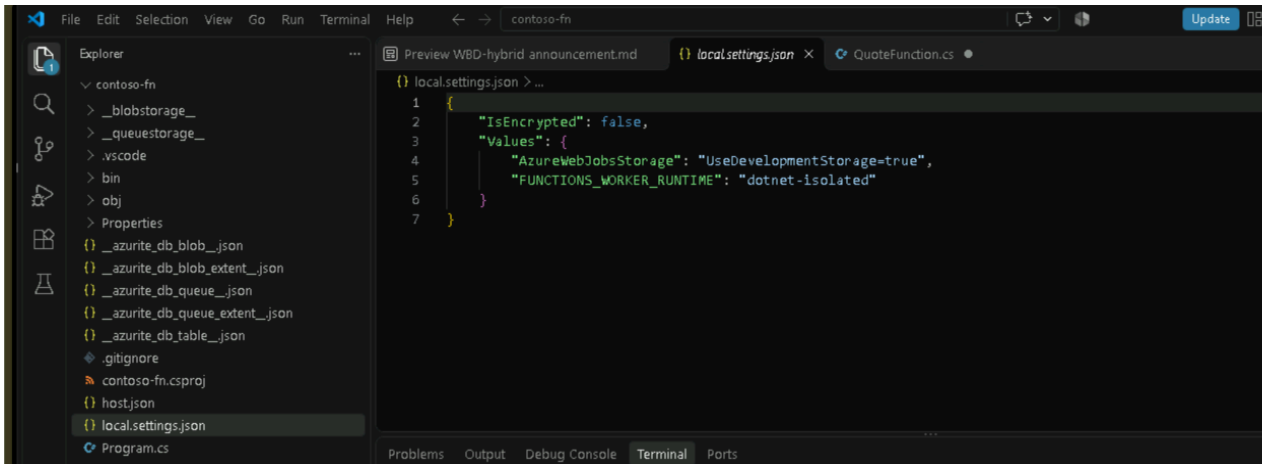


Step 3 – Configure `local.settings.json`

Open the existing `local.settings.json` file and verify the following configuration:

```
None
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage":
"UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated"
  }
}
```

Save the file.

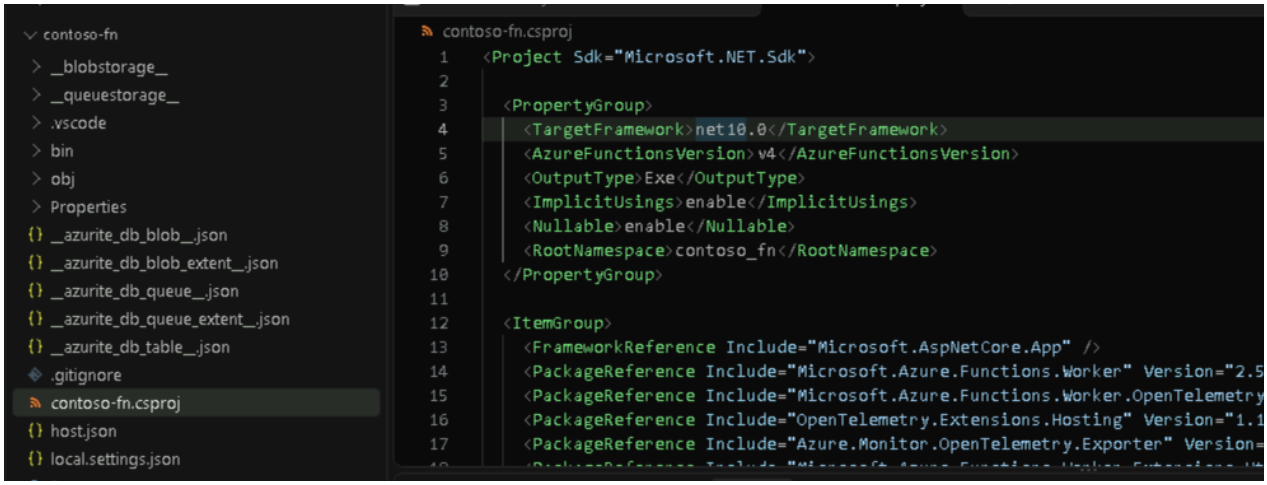


Important: `local.settings.json` is for local development. Never commit it to source control if it contains secrets. The generated `.gitignore` should exclude this file.

Step 4 – Update `contoso-fn.csproj`

Open the existing `contoso-fn.csproj` file and verify the Verify that `<TargetFramework>` is set to `net10.0`:

```
None
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <AzureFunctionsVersion>v4</AzureFunctionsVersion>
  <OutputType>Exe</OutputType>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>
</PropertyGroup>
```



6. Build the Project

Return to the VS Code terminal.

Execute:

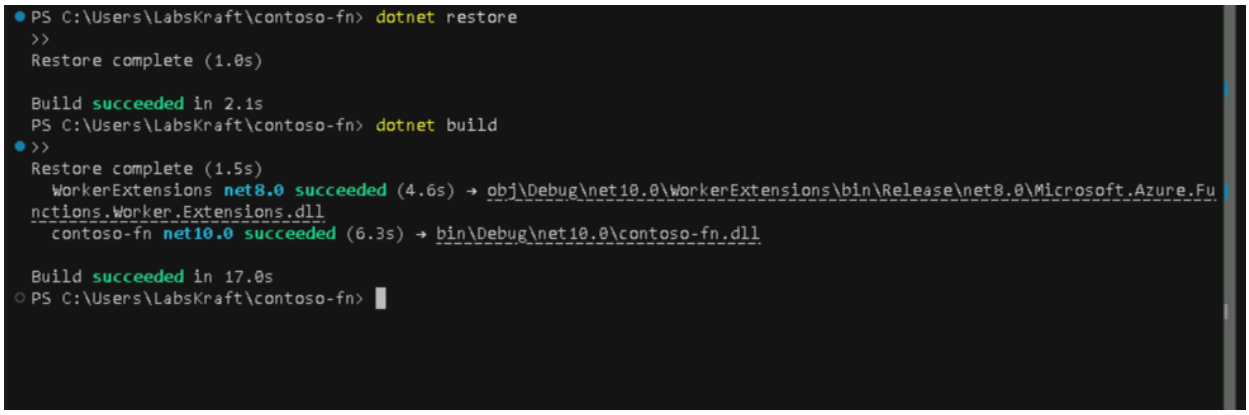
```
None
dotnet restore
```

This restores the project dependencies.

Next, execute:

```
None
dotnet build
```

Expected result: Build succeeds with zero errors.



If the build fails, verify the project targets `net10.0` and that the isolated-worker packages were restored correctly.

7. Create HTTP Test Requests

We will use the VS Code REST Client extension to send requests directly from the editor.

Step 1 – Create a test file

1. In VS Code Explorer, right-click the project folder.
2. Select New File.
3. Name it `requests.http`.

Step 2 – Add the following requests

```
None
### Test 1: Valid order quote
POST http://localhost:7071/api/quote
Content-Type: application/json

{
  "qty": 3,
  "unitPrice": 15.50
}

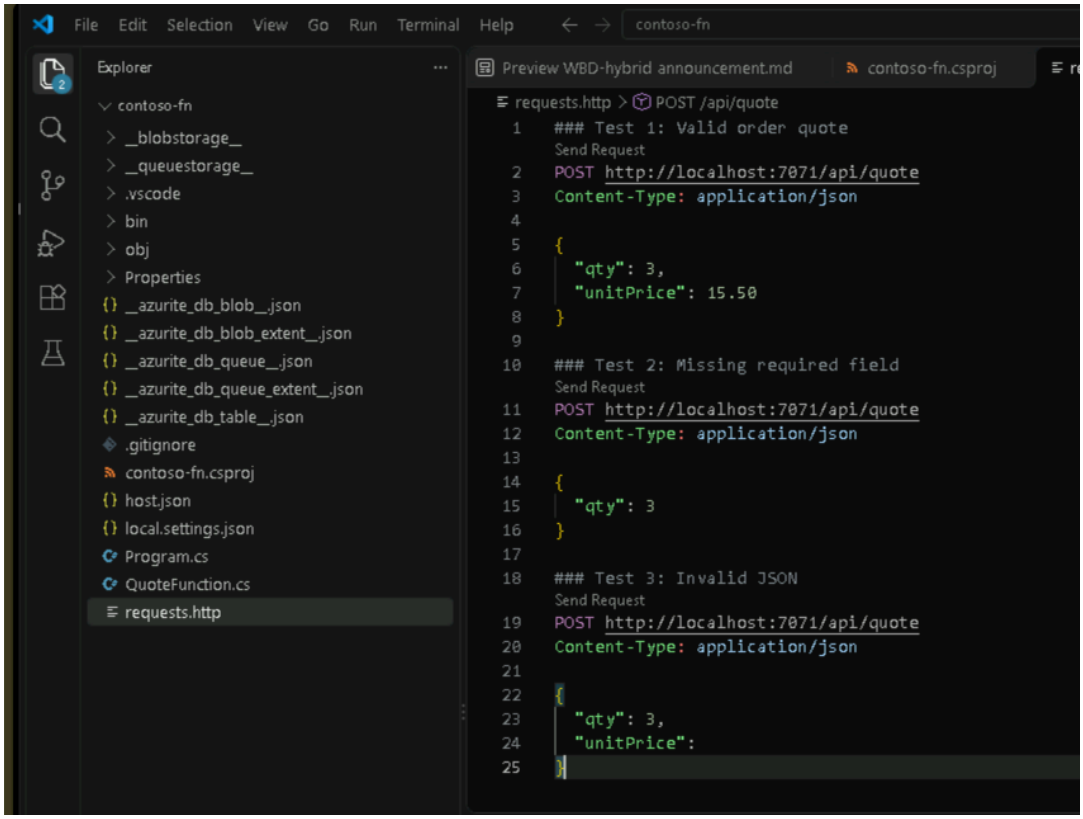
### Test 2: Missing required field
POST http://localhost:7071/api/quote
Content-Type: application/json

{
  "qty": 3
}

### Test 3: Invalid JSON
POST http://localhost:7071/api/quote
Content-Type: application/json

{
  "qty": 3,
  "unitPrice":
}
```

Save the file.



8. Start the Function Locally

Step 1 – Start the Functions runtime

Ensure Azurite is running, then execute from the project directory:

```
None
func start
```

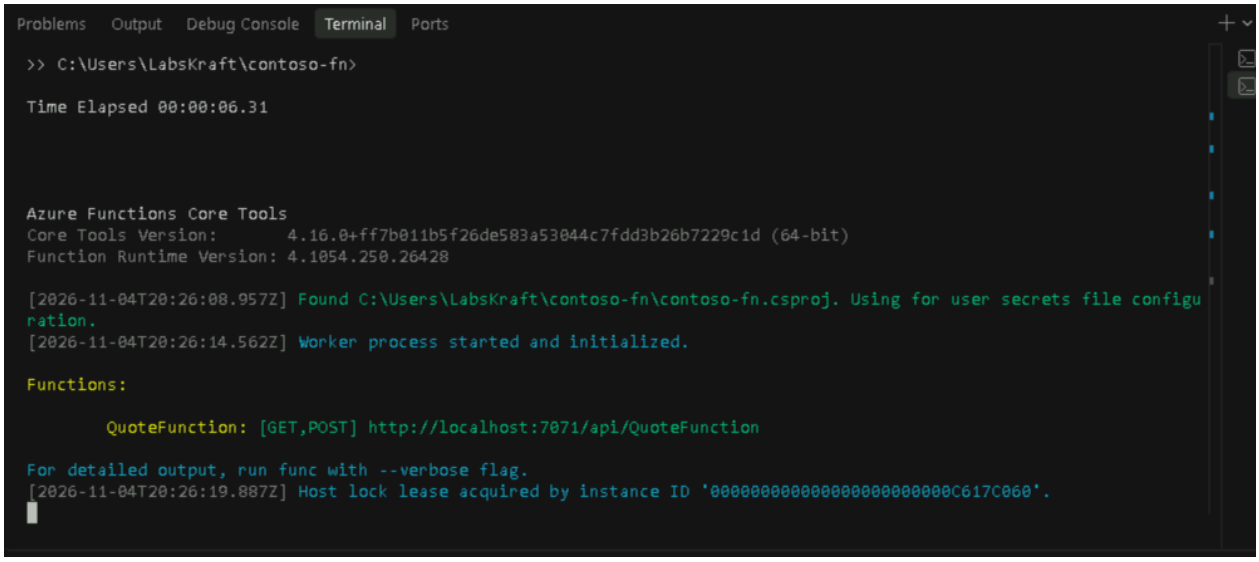
The Functions host should start and display an endpoint similar to:

```
None
Functions:

    QuoteFunction:
```

```
    [POST] http://localhost:7071/api/quote

Job host started
```



Step 2 – Execute the test requests

1. Open `requests.http`.
2. Locate Test 1.
3. Select Send Request above the request.
4. Inspect the response.
5. Repeat for Test 2 and Test 3.

9. Validate the API Responses

Test 1 – Valid request

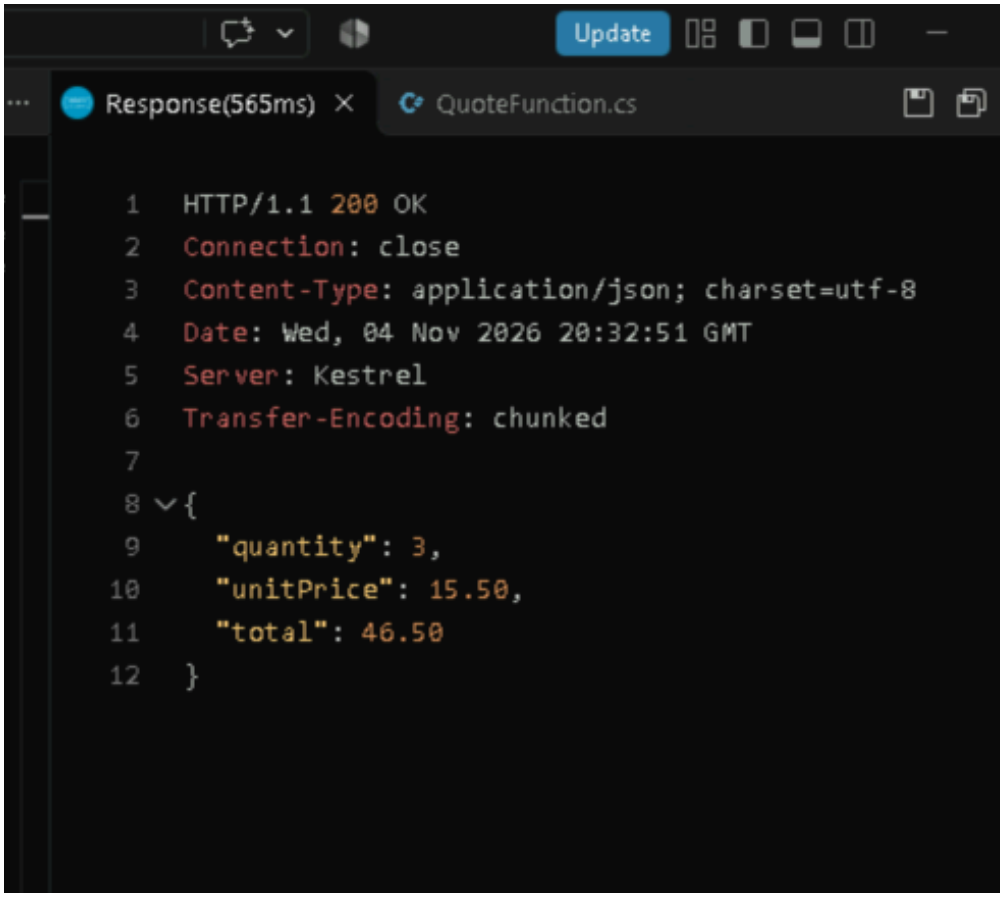
Input:

```
None
{
  "qty": 3,
  "unitPrice": 15.50
}
```

Expected response:

HTTP 200 OK

```
None
{
  "quantity": 3,
  "unitPrice": 15.50,
  "total": 46.50
}
```



Test 2 – Missing required field

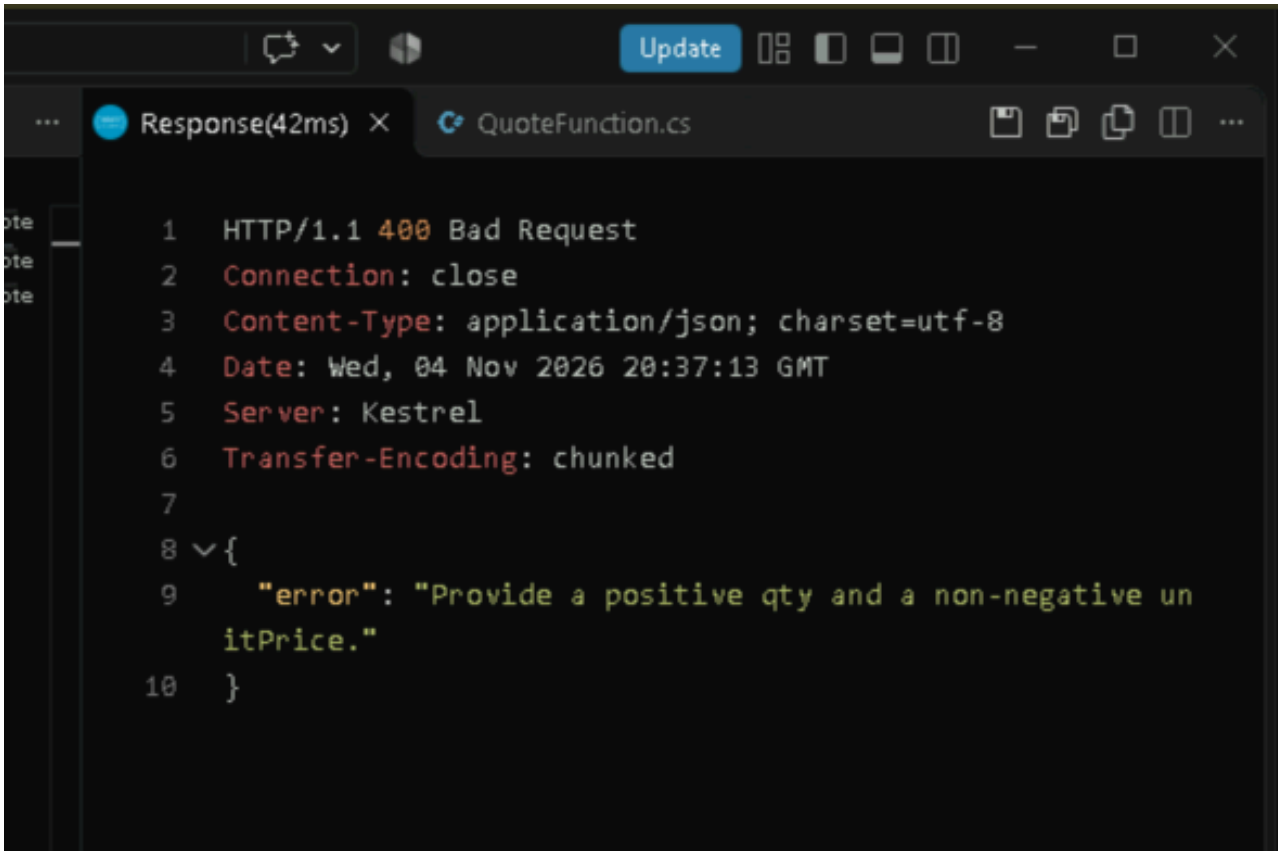
Input:

```
None
{
  "qty": 3
}
```

Expected response:

HTTP 400 Bad Request

```
None
{
  "error": "Provide a positive qty and a non-negative unitPrice."
}
```



Test 3 – Invalid JSON

Input:

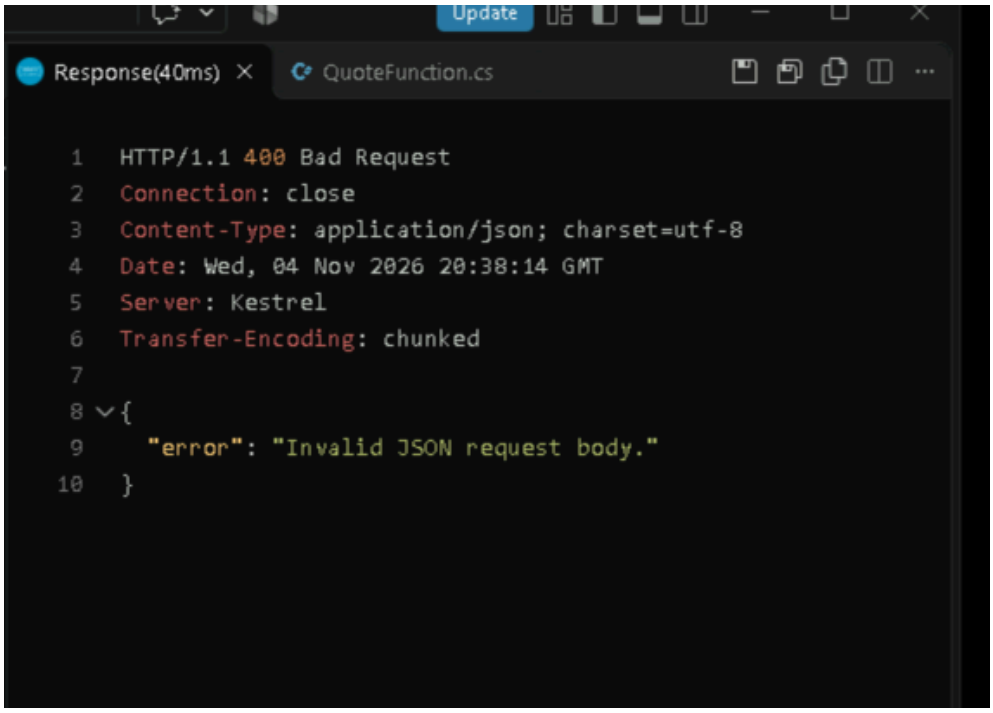
```
None
{
  "qty": 3,
```

```
  "unitPrice":
}
```

Expected response:

HTTP 400 Bad Request

```
None
{
  "error": "Invalid JSON request body."
}
```



10. Verify Function Authorization

Open `QuoteFunction.cs`.

Locate:

```
None
AuthorizationLevel.Function
```

This setting requires a Function key when the Function is hosted in Azure.

For local execution, Azure Functions Core Tools disables HTTP authorization by default. Therefore, you can call the endpoint locally without a Function key, even with `AuthorizationLevel.Function`.

To test local key enforcement, you can optionally start the host using:

```
None
func start --enableAuth
```

This requires a recent Core Tools v4 release.

Microsoft Learn+1

We will verify actual Function key enforcement after deployment in Lab 1.3.

11. Troubleshooting

Issue	Resolution
`func` command not found	Verify Core Tools installation and PATH.

Port 7071 already in use	Stop the other Functions host or use another port.
Azurite connection failure	Start Azurite and verify its local services.
Function not discovered	Check `[Function("QuoteFunction")]` and rebuild.
Build errors	Verify .NET 10 SDK and project dependencies.
REST Client not visible	Install and enable the REST Client extension.
HTTP 404	Verify the route and `/api/quote` endpoint.

12. Acceptance Criteria

The lab is complete when:

- C# isolated-worker project is created successfully.
- HTTP-triggered Function is generated.
- Order Quote API implementation builds successfully.

- Function starts using Core Tools.
- Valid request returns HTTP 200.
- Missing field returns HTTP 400.
- Malformed JSON returns HTTP 400.
- Function authorization behaviour is understood.